

Professional Coding Specialist

COS Pro 파이썬 2 급

24 강-26 강. 모의고사 6 차

모의고사 6 차 문제 1-10 번

과정 소개

모의고사(YBM 샘플) 문제를 통해 실전에 대비할 수 있다. (회차당 10 문항, 각 문항별 문제 풀이)

학습 목차

모의고사 6 차 문제 1 번
모의고사 6 차 문제 2 번
모의고사 6 차 문제 3 번
모의고사 6 차 문제 4 번
모의고사 6 차 문제 5 번
모의고사 6 차 문제 6 번
모의고사 6 차 문제 7 번
모의고사 6 차 문제 8 번
모의고사 6 차 문제 9 번
모의고사 6 차 문제 10 번

학습 목표

1. YBM IT(www.ybmit.com) 에서 제공하는 COS Pro 파이썬 2 급 샘플 문제 메뉴의 모의고사를 풀어보며 COS Pro 파이썬 2 급 시험에 대비할 수 있다.

1

모의고사 6 차 문제 1 번

문제 1 의 정답

두 날짜보다 기온이 높았던 날의 수

```
def solution(temperature, A, B):
    answer = 0
    ① for i in range(A+1, B):
        ② if temperature[i] > temperature[A] \
           and temperature[i] > temperature[B]:
            answer += 1
    return answer
```

	A					B		
	0	1	2	3	4	5	6	7
	3	2	1	5	4	3	3	2

① A 와 B 날짜 사이에서 for 문을 반복해야 하므로 A+1, B 로 적어야 한다. 시작에 A 가 포함되지 않으므로 A+1 로 하는 것에 유의한다.

② 현재 가져온 기온 데이터 temperature[i] 가 A, B 번째 기온인 temperature[A], temperature[B]의 데이터보다 크면 두 날짜보다 기온이 높았던 날이므로 날의 수인 answer 변수의 값을 증가시킨다.

문제 2 의 정답

필요한 만큼 종이를 받은 사람의 수 구하기

```
def solution(papers, K):
    length = len(papers)
    ① for i, paper in enumerate(papers):
        ② K -= paper
        if K < 0:
            return i
    break
    return length
```

사람별 필요한 종이 수	총 종이 수	필요한 만큼 종이를 가져간 사람 수
papers	K	return
[2, 4, 3, 2, 1]	10	3
[2, 4, 3, 2, 1]	14	5

필요한 만큼 종이를 받은 사람 수 (마지막 인덱스 번호)

0	1	2	3	4
2	4	3	2	1

남은 종이 수 < 0

- ① 사람별 필요 종이 수가 담긴 리스트(papers)를 읽어 인덱스 번호(i)와 데이터 값(paper)을 가져온다.
- ② 총 종이 수(K)에서 각 사람들이 필요한 종이 수(paper)를 차감시킨다.
 남은 종이의 값이 0 보다 작으면 인덱스 번호(i)를 리턴한다.
 이 문제는 한 줄 수정하기 문제이므로 length=i 문장 대신 return i 로 수정한다.
 하지만 한 줄을 추가해도 된다면 length=i 문장 뒤에 break 를 적어주면 된다.

3

모의고사 6 차 문제 3 번

문제 3 의 정답

주문해야 하는 유니폼 사이즈 수

```
def solution(people):
    answer = [0 for _ in range(4)]

    for p in people:
        if p < 95:
            answer[0] += 1
        elif 95 <= p < 100:
            answer[1] += 1
        elif 100 <= p < 105:
            answer[2] += 1
        else:
            answer[3] += 1

    return answer
```

덩치	사이즈
95 미만	S
95 이상 100 미만	M
100 이상 105 미만	L
105 이상	XL

people	return
[97, 102, 93, 100, 107]	[1, 1, 2, 1]
M, L, S, L, XL	[S, M, L, XL]

사람들의 사이즈(people)를 읽어서 사이즈 표를 참고하여 사이즈별로 개수를 저장한다. 최종 결과 리스트는 [S, M, L, XL] 순으로 저장해야 하는 것에 유의한다.

문제 4 의 정답

카드의 총 점수 계산

```
def solution(cards):
    answer = 0
    1 color = [0 for _ in range(3)]
    2 for card in cards:
        if card[0] == 'black':
            color[0] += 1
        elif card[0] == 'blue':
            color[1] += 1
        elif card[0] == 'red':
            color[2] += 1
        answer += int(card[1])
    3 if color[0] == 3 or color[1] == 3 or color[2] == 3:
        answer *= 3
    elif color[0] == 2 or color[1] == 2 or color[2] == 2:
        answer *= 2
    return answer
```

card	[0], [1]	cards	return
	["blue", "2"]	["red", "5"], ["black", "3"]	10
	["blue", "2"]	["blue", "5"], ["black", "3"]	20

- ① 색상별 카드의 개수를 저장할 변수(color)를 0 으로 초기화 한다.
- ② 2 차원 리스트 cards 의 데이터를 1 차원 형태로 하나씩 읽어 와서 card 변수에 넣는다.
(색상 : card[0], 점수 : card[1])
card[0]에 색상 정보가 들어오게 되므로 card[0]에 들어 있는 “black”, “blue”, “red” 값을 이용해 각각 color 의 0 번, 1 번, 2 번 인덱스에 색상별 카드 개수를 구한다.
카드의 총 점수를 합산하기 위해서 card[1]에 들어 있는 데이터를 이용한다.
- ③ 점수 계산 규칙에 맞게 총 점수를 재계산한다.
 1. 카드 3 장의 색이 모두 같다면 → 총합에 3 을 곱한 값
 2. 카드 2 장의 색이 같다면 → 총합에 2 를 곱한 값
 3. 카드 3 장의 색이 모두 다르다면 → 총합

[2 급 6 차 4 번 다른 코딩 제안-1.py]

```
def solution(cards):
    answer = 0
    color = [0 for _ in range(3)]
    for card in cards:
        if card[0] == 'black':
            color[0] += 1
        elif card[0] == 'blue':
            color[1] += 1
        elif card[0] == 'red':
            color[2] += 1
        answer += int(card[1])
    if '3' in map(str,color):
        answer *= 3
    elif '2' in map(str,color):
        answer *= 2
    return answer
```

[2 급 6 차 4 번 다른 코딩 제안-2.py]

```
def solution(cards):
    answer = 0
    color={"red":0,"black":0,"blue":0}

    for i in range(len(cards)):
        color[cards[i][0]]+=1
        answer += int(cards[i][1])

    if max(color.values())==3:
        answer=answer*3
    elif max(color.values())==2:
        answer=answer*2
    return answer
```

5

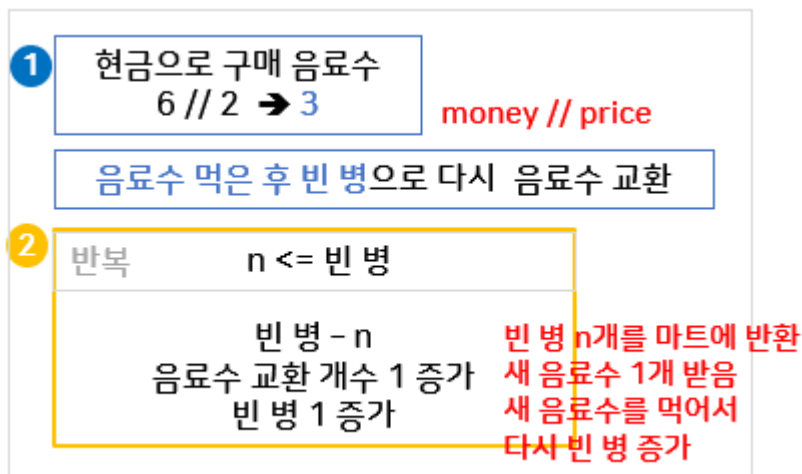
모의고사 6 차 문제 5 번

문제 5 의 정답

마실 수 있는 총 음료수의 수

```
def solution(money, price, n):
    answer = 0
    1 empty_bottle = answer = money // price
    while n <= empty_bottle:
        empty_bottle = empty_bottle - n
        2 answer += 1
        empty_bottle += 1
    return answer
```

현재 가진 돈	음료수 1병 가격	빈 병 n개당 음료수 1병 교환	마실 수 있는 총 음료수
money	price	n	return
8	2	4	5
6	2	2	5



- ① 현금으로 구매할 수 있는 음료수의 개수를 센다.

$money // price$ 즉 현재 가진 돈($money$)을 음료수 1 병의 가격($price$)으로 나눈 몫을 이용하면 된다. 구매한 음료수를 먹으면 다시 빈 병이 되므로 `empty_bottle` 변수에 구매한 음료수의 개수를 집어 넣는다.

- ② 빈 병으로 음료수를 교환할 수 있다면 빈 병 n 개를 마트에 반환하고, 새 음료수를 1 개 받는다. 그리고 새로 받은 음료수가 다시 빈 병이 되므로 빈 병을 하나 증가시킨다. 남은 빈 병으로 음료수를 교환할 수 있는 동안 해당 과정을 반복한다.

이 문제에서는 빈 병이 생기면 빈 병 n 개를 마트에 갖다 주기 때문에 본인이 가진 빈 병의 개수는 감소하게 된다. 따라서 n 개 만큼의 병이 감소하기 때문에 `empty_bottle = empty_bottle + n` 문장에서 `+`를 `-`로 수정한다.

`empty_bottle = empty_bottle - n` → 빈병 n 개를 마트에 반환

`answer += 1` → 빈 병을 마트에 주고 새 음료수를 1 개 받음

`empty_bottle += 1` → 빈 병을 교환하여 받은 새 음료수를 마시면 다시 빈 병이 되므로 빈 병 증가

이 문제에서는 교환한 음료수가 다시 빈 병이 되는 것에 주의해야 한다.

6

모의고사 6 차 문제 6 번

문제 6 의 정답

생성 가능한 비밀번호 판단

```
def solution(password):
    capital_count, small_count, digit_count = 0, 0, 0
    1 for p in password:
        2 if p >= 'A' and p <= 'Z':
            capital_count += 1
        elif p >= 'a' and p <= 'z':
            small_count += 1
        elif p >= '0' and p <= '9':
            digit_count += 1
    3 if @@@:
        answer = True
    else:
        answer = False
    return answer
```

capital_count >= 1
and small_count >= 2
and digit_count >= 2

<비밀번호 생성 규칙>

1. 한 개 이상의 알파벳 대문자
2. 두 개 이상의 알파벳 소문자
3. 두 개 이상의 숫자

- ① password의 글자를 하나씩 읽어서 p 변수에 넣는다.
 ② 비밀번호 생성 규칙에 맞게 알파벳 대문자(capital_count), 알파벳 소문자(small_count), 숫자(digit_count) 변수에 각각의 개수를 센다.
- ② 알파벳 대문자 1 개 이상 and 알파벳 소문자 2 개 이상 and 숫자 2 개 이상이면 True 아니면 False 를 리턴한다.

문제 7 의 정답

예산 내에서 의자와 책상 가격 합의 최댓값

```
def solution(money, chairs, desks):
    answer = 0
    ① for chair in chairs:
        ② for desk in desks:
            ③ price = chair + desk
            if answer < price and @@@:
                answer = price
    return answer
```

price <= money

의자와 책상의 모든 조합 중 예산을 넘지 않으면서 최댓값을 찾는 문제이다.

① 의자(chairs)의 데이터 값을 하나씩 가져와서 chair 변수에 넣는다.

② 책상(desks)의 값을 하나씩 가져와서 desk 변수에 넣는다.

③ 의자와 책상의 모든 조합을 비교한다.

합(price) = 의자 가격 + 책상 가격

현재 최댓값(answer)보다 합(price)이 크고 합(price)이 예산(money) 이하이면

최댓값(answer)을 현재 합(price)으로 변경한다.

8

모의고사 6 차 문제 8 번

문제 8 의 정답

수와 뒤집은 수의 차 구하기

```
def func_a(number1, number2):
    3 ret = 0
    if number1 > number2:
        ret = number1 - number2
    else:
        ret = number2 - number1
    return ret
```

```
def func_b(number):
    1 ret = 0
    while number != 0:
        number = number // 10
        ret += 1
    return ret
```

```
def func_c(number, digit):
    2 ret = 0
    for i in range(digit):
        temp = number % 10
        number = number // 10
        ret = ret * 10 + temp
    return ret
```

```
def solution(number):
    answer = 0
    digit = func_b(number)
    convert_number = func_c(number, digit)
    answer = func_a(number, convert_number)
    return answer
```

예를 들어, 주어진 수가 120 인 경우 이를 뒤집으면 021 입니다.
십진수에서 앞의 0 은 제거해야 하므로 120 을 뒤집은 수는 21 입니다.

① 주어진 수의 총 자릿수를 구한다.

수 // 10 을 이용하여 더 이상 숫자가 없을 때까지 count 를 하여 총 자릿수를 구한다.

```
def func_b(number):
    ret = 0
    while number != 0:
        number = number // 10
        ret += 1
    return ret
```

- ② 구한 자릿수를 이용하여 주어진 수를 뒤집는다.

```
def func_c(number, digit):  
    ret = 0  
    for i in range(digit):  
        temp = number % 10  
        number = number // 10  
        ret = ret * 10 + temp  
    return ret
```

맨 뒷자리 수를 찾기 위해서 숫자를 10으로 나눈 나머지를 이용한다. (수%10)

맨 뒷자리 수를 없애고 앞 자리 수들만 남기기 위해 숫자를 10으로 나눈 몫을 이용한다. (수//10)

맨 뒷자리 수부터 계속해서 자릿수 *10 을 함으로써 숫자를 거꾸로 뒤집게 된다.

위 문제와 같이 함수를 구현해도 되지만, 아래와 같이 숫자를 문자열로 바꾼 후 len()함수를 이용하여 숫자의 자릿수를 구해도 된다.

```
digit= len(str(number))
```

- ③ 주어진 수와 뒤집은 수의 차이 구하기

```
def func_a(number1, number2):  
    ret = 0  
    if number1 > number2:  
        ret = number1 - number2  
    else:  
        ret = number2 - number1  
    return ret
```

주어진 수와 뒤집은 수의 차이를 구하는데 어떤 숫자가 큰 지 확인하여 큰 수에서 작은 수를 뺀다.

아래와 같은 방식으로 abs()함수를 사용해도 된다.

```
abs(number-int(convert_number))
```

[2 급 6 차 8 번 다른 코딩 제안.py]

```
def solution(number):  
    answer = 0  
    convert_number=''  
    digit=len(str(number))  
  
    for i in range(digit-1,-1,-1):  
        convert_number+=str(number)[i]  
  
    answer=abs(number-int(convert_number))  
  
    return answer
```

※ str() 함수와 abs() 함수를 이용하면 위와 같은 코드도 가능하니 참고해 보세요.

문제 9 의 정답

만들 수 있는 양말 쌍의 개수

```
def solution(socks):
    answer = 0
    count = [0 for _ in range(10)]
    ① for s in socks:
        count[s] += 1
    ② for c in count:
        answer += (c // 2)
    return answer
```

socks	return
[1, 2, 1, 3, 2, 1]	2

숫자가 색깔을 의미
하므로 숫자별 개수
를 합산해서 2개당 1
쌍을 만든다.

count

0	1	2	3	...
0	3	2	1	

- ① 색깔별 양말의 총 개수를 구한다.
socks 에서 데이터를 하나씩 읽어 와서 해당 숫자의 인덱스 번호의 개수를 증가한다.
- ② 양말 2 개당 1 쌍으로 만든다.
색깔별 양말의 총 개수가 들어 있는 count 리스트의 데이터를 읽어 와서 양말 2 개당 양말 1 쌍을 만들어 총 만들 수 있는 쌍의 수를 구한다. 이때 2 개당 한 쌍을 만드는 수식으로 2 로 나눈 몫을 이용하면 된다.

10

모의고사 6 차 문제 10 번

문제 10 의 정답

불량품 개수 구하기

```
def solution(weight, boxes):
    answer = 0
    1 for x in boxes:
        2 if @@@:
            answer += 1
    return answer
```

$x < \text{weight} * 0.9$ or $x > \text{weight} * 1.1$

weight	boxes	return
600	[653, 670, 533, 540, 660]	2

540 미만 , 660 초과 이면 불량

- ① boxes 에서 데이터를 하나씩 가져온다.
 ② weight 의 $\pm 10\%$ 이내가 아니면 (-10% 미만 이거나 $+10\%$ 초과) 불량품 개수 증가시킨다.